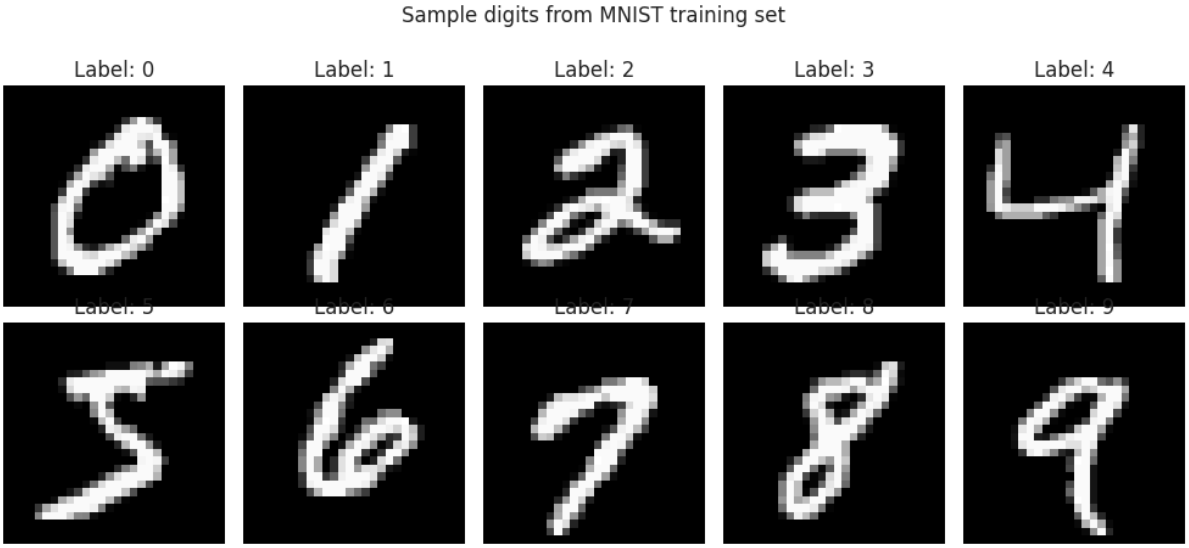
Neural Network for Handwritten Digit Recognition Using MNIST

Introduction

The MNIST dataset is one of the most widely used benchmarks in computer vision (LeCun et al., 1998). It contains 70,000 grayscale images of handwritten digits from 0 to 9, each 28 by 28 pixels, with 60,000 images for training and 10,000 for testing. The dataset is simple by today's standards, but it is still useful for teaching because it reduces object recognition to the basics. The model has to recognise shapes that vary in stroke thickness, slant, and personal handwriting style, with no colour, texture, or background to distract it.
Handwritten digit recognition is also one of the original applied success stories of neural networks. The work that introduced MNIST was driven by a real-world need — automating the reading of postal codes and bank cheques (LeCun et al., 1998). The clean structure of the problem (ten classes, fixed image size, lots of labelled examples) made it a useful benchmark, and it has stayed in use ever since because it lets students and researchers test ideas quickly. In this essay I describe the design of a convolutional neural network (CNN) for MNIST, the reasoning behind each choice, and what I learned from building it.



Caption: Figure 1: Sample digits from the MNIST training set, one per class.

Data preparation and validation set
MNIST is already split into 60,000 training images and 10,000 test images. To check the model during training without touching the test set, I split the training data again into 80% training and 20% validation. This gave me 48,000 images for training, 12,000 for validation, and 10,000 held out for the final test. The split was random but used a fixed random seed, so the same images go into the same set each time the code runs. This kind of reproducibility matters because it lets me compare results between runs without worrying that a change in performance is just down to a different shuffle.

Pixel values were normalised by dividing by 255. This maps each pixel from the original 0–255 range to a 0–1 range. Neural networks learn more reliably with small input numbers because gradient updates stay in a manageable range, which prevents the optimiser from overshooting (Goodfellow, Bengio and Courville, 2016). Without normalisation, large input values can produce large activations, which in turn produce large gradients, and training can become unstable. The labels were kept as integer class indices from 0 to 9, which works directly with the loss function I describe later.

The validation set is essential. Without it, the only signal during training is the training loss itself, which always falls as the model fits the data more tightly — including fitting noise that is specific to that data. The validation set gives an honest signal: how well the model performs on images it has not seen. If training accuracy keeps rising while validation accuracy stops improving, the model is overfitting and training should stop. Without a validation set, this is invisible until the test set is run, by which point it is too late to fix.

The test set has a different role again. It is used only once, at the very end, to give a final estimate of how well the model generalises. The discipline of keeping these three sets separate is what makes the reported accuracy honest. If the test set is used during development — even just for choosing hyperparameters — its accuracy estimate becomes optimistic, because the model has effectively been tuned to it.

Network architecture

The network is a small CNN with the following structure: a convolutional layer with 32 filters of size 3×3, a 2×2 max pooling layer, a second convolutional layer with 64 filters of size 3×3, another 2×2 max pooling layer, a flatten layer, and finally two fully connected (dense) layers — one with 64 units and an output layer with 10 units, one for each digit class.

CNNs are a good choice for image data because of how convolutional layers work. Each filter slides across the image and learns to respond to a particular local pattern — an edge, a curve, a corner. Early layers learn simple shapes, and deeper layers combine these into more complex features such as loops or junctions, which are the building blocks of digits (LeCun et al., 1998). A simpler fully connected network would also work for MNIST and reach around 97% accuracy, but it would treat every pixel as independent and ignore the spatial structure of the image. The CNN uses that structure directly, which improves both accuracy and efficiency.

The number of filters increases with depth — from 32 in the first convolutional layer to 64 in the second. This is a common pattern in CNNs. Early layers only need to detect a small set of low-level features (edges in different directions, simple curves), so 32 filters are enough. Deeper layers combine those low-level features into more specific patterns, and there are many more possible combinations, so it makes sense to give the network more filters at that point. Each filter is small (3×3) because small filters with several layers stacked together can capture the same spatial range as a larger filter, but with fewer parameters and more non-linearity in between.

The pooling layers reduce the size of the feature maps after each convolutional block. A 2×2 max pooling layer takes each 2×2 patch in the feature map and keeps only the largest value, which halves the height and width. This reduces computation in later layers and gives the network some translation invariance — small shifts in where the digit is drawn within the image do not change the output much, because the same pattern still ends up as the maximum in nearby regions. The flatten layer reshapes the final feature maps into a single vector, so the dense layers can produce a final classification.

Activation functions

The hidden layers use the ReLU (Rectified Linear Unit) activation function, defined as ReLU(x) = max(0, x). ReLU is simple, fast to compute, and avoids the vanishing gradient problem that affects sigmoid and tanh activations in deeper networks (Nair and Hinton, 2010). Sigmoid and tanh squash their outputs into a fixed range, and their gradients become very small for large positive or negative inputs. When several layers are stacked, these small gradients multiply together during backpropagation and become close to zero, which means the early layers stop learning. ReLU does not have this problem on the positive side, because its gradient is always one when the input is positive. The learning signal passes through unchanged, even in deeper networks.

ReLU does have one weakness — neurons can sometimes get stuck outputting zero (the "dying ReLU" problem) if their weights end up in a state where the input is always negative. In practice this rarely causes major issues at this scale, but it is something to be aware of when building deeper networks.

The output layer uses softmax activation. Softmax converts the ten raw output values (logits) into a probability distribution that sums to one. Each output is exponentiated and then divided by the sum of all the exponentiated outputs. This means a slightly higher logit becomes a much higher probability, which makes softmax good at expressing confident predictions. It also lets the network express how confident it is across all ten classes at once, not just its top guess. Softmax pairs naturally with the cross-entropy loss function used during training.

Loss function

The loss function is sparse categorical cross-entropy. Cross-entropy measures the difference between the predicted probability distribution and the true class label. If the network predicts a probability of 0.9 for the correct class, the loss is small. If it predicts 0.1 for the correct class, the loss is large. This penalises confident mistakes more than uncertain ones, which encourages the network to stay calibrated. A model that says "I think this is a 7 with 60% confidence" and is wrong is treated more gently than one that says "I am 99% sure this is a 7" and is wrong.

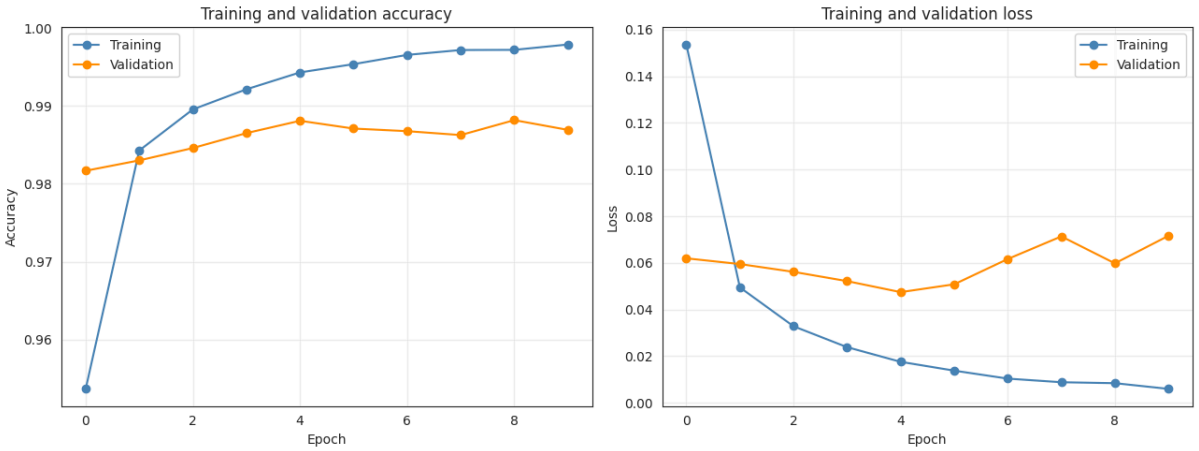
The "sparse" version is used when labels are stored as integer class indices (0 to 9) instead of one-hot vectors. It produces the same result as standard categorical cross-entropy, but uses less memory and avoids an extra preprocessing step. Minimising this loss during training pushes the network's output probabilities towards the correct class for each image.

Training strategy: epochs, batch size, and optimiser

The model was trained for 10 epochs. One epoch means the network sees the full training set once, so 10 epochs means it sees the data ten times. MNIST is a relatively simple dataset and the network converges quickly. By 5 epochs the validation accuracy is already close to its final value, and additional epochs give smaller and smaller improvements. I chose 10 as a safe upper bound that gives the network room to settle without wasting compute or risking serious overfitting.

A batch size of 32 was used. This means the model processes 32 images at a time, computes the average loss across those 32 images, and updates its weights based on that average. Smaller batches give noisier gradient updates, which can help the model escape local minima but make training slower per epoch. Larger batches give smoother updates but use more memory and can sometimes lead to worse generalisation. 32 is a common default that works well for MNIST and balances these trade-offs.

The optimiser is Adam (Kingma and Ba, 2014). Adam combines two ideas. The first is momentum, which smooths gradient updates over time — instead of stepping in the direction of the current gradient, the optimiser keeps a running average of recent gradients and steps in that direction. This helps it move steadily through flat regions and avoid getting stuck. The second is adaptive learning rates, which scale updates per parameter based on recent gradient history. Parameters that have had large gradients get smaller updates, and parameters with small gradients get larger updates. This means each parameter effectively has its own learning rate, which makes Adam less sensitive to the initial choice of learning rate than simpler optimisers like stochastic gradient descent. For these reasons Adam is widely used as a default optimiser, because it works well across many problems without much tuning. The metric tracked during training is accuracy, which is intuitive and works well for a balanced classification task like MNIST.



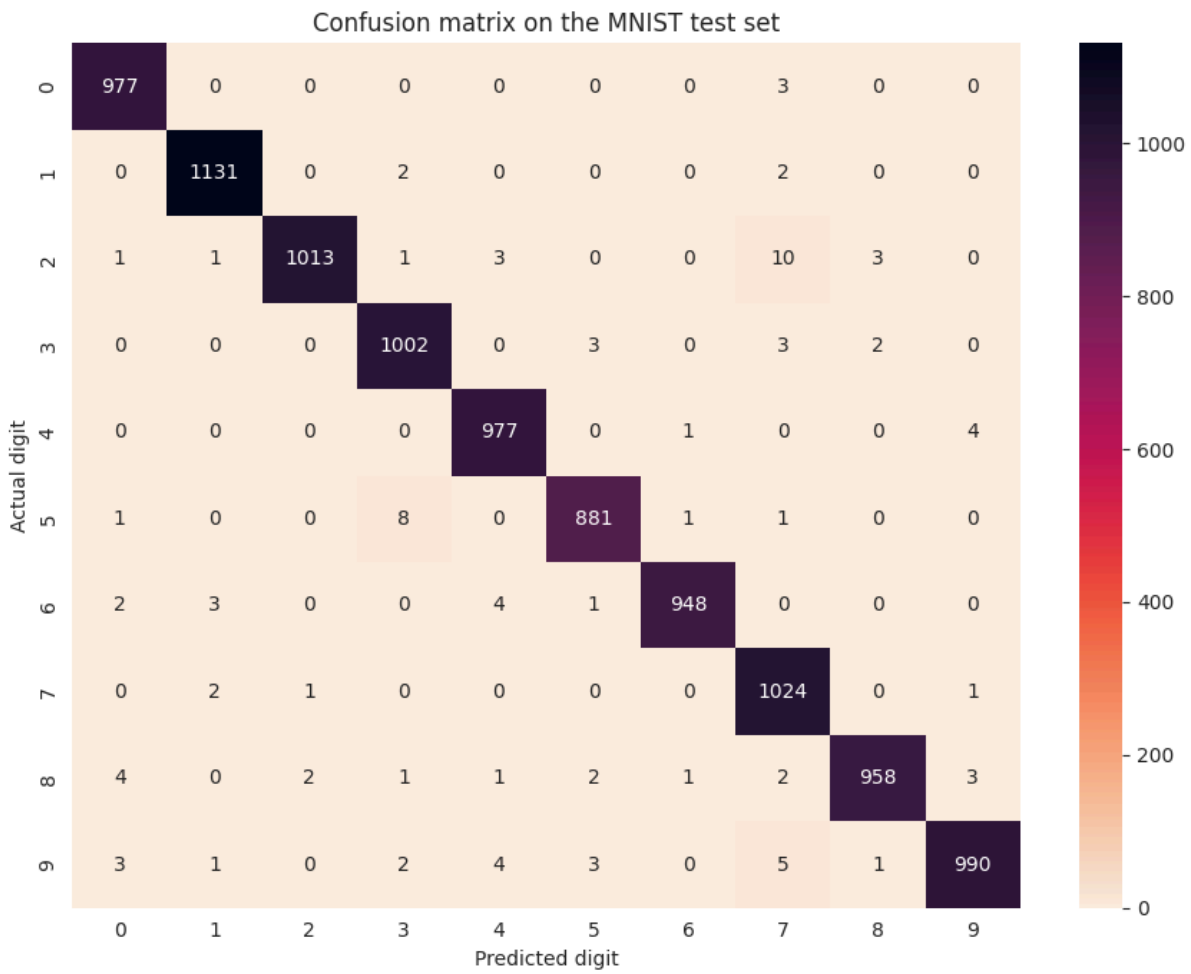
Caption: Figure 2: Training and validation accuracy and loss across 10 epochs.

Reflections and learnings

The model reached 99.01% accuracy on the test set. The most important practical lesson from this work was the value of a proper train/validation/test split. Without a separate validation set, there is no honest way to tune a model. Every adjustment based on the test set leaks information and makes the final accuracy estimate too optimistic. The discipline of leaving the test set untouched until the very end is what makes the reported accuracy trustworthy. This applies far beyond MNIST — any machine learning project needs to design its data split carefully before any modelling begins, because changes made later are much harder to undo.

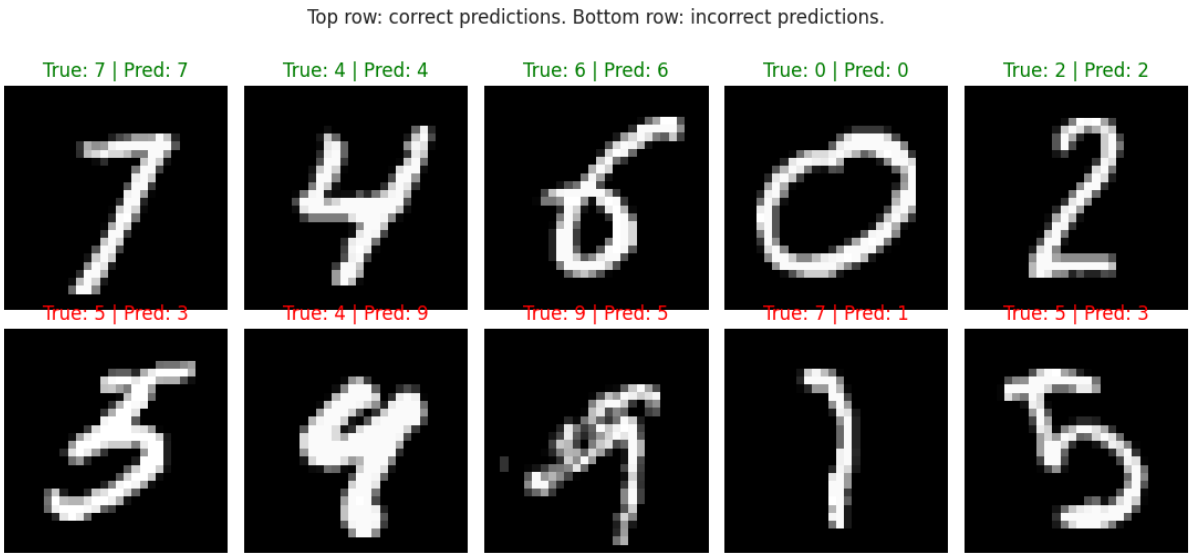
The training curves also showed a mild overfitting trend after about epoch 5, where validation loss began to rise slightly while training loss kept falling. The model still generalised well to the test set, but this is a useful warning sign. In a longer training run, it would have got worse. Future iterations could benefit from early stopping (stopping training automatically when validation loss stops improving) or regularisation techniques like dropout (which randomly switches off some neurons during training, forcing the network to not depend too much on any single feature). Both would help keep validation loss in line with training loss for longer.

The second lesson is about model limitations. Even at 99% accuracy, the model still misclassifies around 100 images in the test set. Looking at the confusion matrix shows that common confusions include 2s being misclassified as 7s, 5s as 3s, and 4s as 9s — visually similar digits that humans also occasionally confuse.



Caption: Figure 3: Confusion matrix on the MNIST test set. Diagonal cells show correct predictions; off-diagonal cells show misclassifications.

Looking at examples of the actual misclassified images is also revealing. Many of the wrong predictions are on digits that are actually hard to tell apart — a 5 that looks like a 3 because of how it was written, or a 9 with an unusual loop that resembles a 4. These are mistakes that humans also occasionally make, which suggests the model is close to what is realistically possible on this dataset, rather than failing at something a more sophisticated network would solve easily.



Caption: Figure 4: Example correct predictions (top row) and incorrect predictions (bottom row). Many incorrect cases involve genuinely unclear handwriting.

A third lesson is more general. Building this network felt like making a series of small, defensible choices rather than one big design decision. The architecture, the activation functions, the loss, the optimiser, the batch size — each one has a reason behind it, and each one could have been chosen differently with a different trade-off. Understanding the reasons matters more than memorising the defaults, because the defaults change with new research and new problems. The same kind of thinking applies when building any machine learning system: the choices need to follow from the problem, not from habit.
Future improvements would most likely come from data augmentation (small rotations and shifts to handle handwriting variance), regularisation methods such as dropout, or extending the network to harder variants such as Fashion-MNIST or EMNIST. Each of these would test the network in a slightly different way and surface different weaknesses, which is the only real way to keep learning beyond a single benchmark.

References

Goodfellow, I., Bengio, Y. and Courville, A. (2016) Deep Learning. Cambridge, MA: MIT Press.

Kingma, D. P. and Ba, J. (2014) ‘Adam: A method for stochastic optimization’, arXiv:1412.6980.

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998) ‘Gradient-based learning applied to document recognition’, Proceedings of the IEEE, 86(11), pp. 2278–2324.

Nair, V. and Hinton, G. E. (2010) ‘Rectified linear units improve restricted Boltzmann machines’, Proceedings of the 27th International Conference on Machine Learning, pp. 807–814